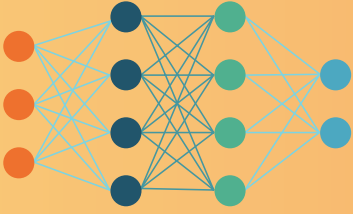
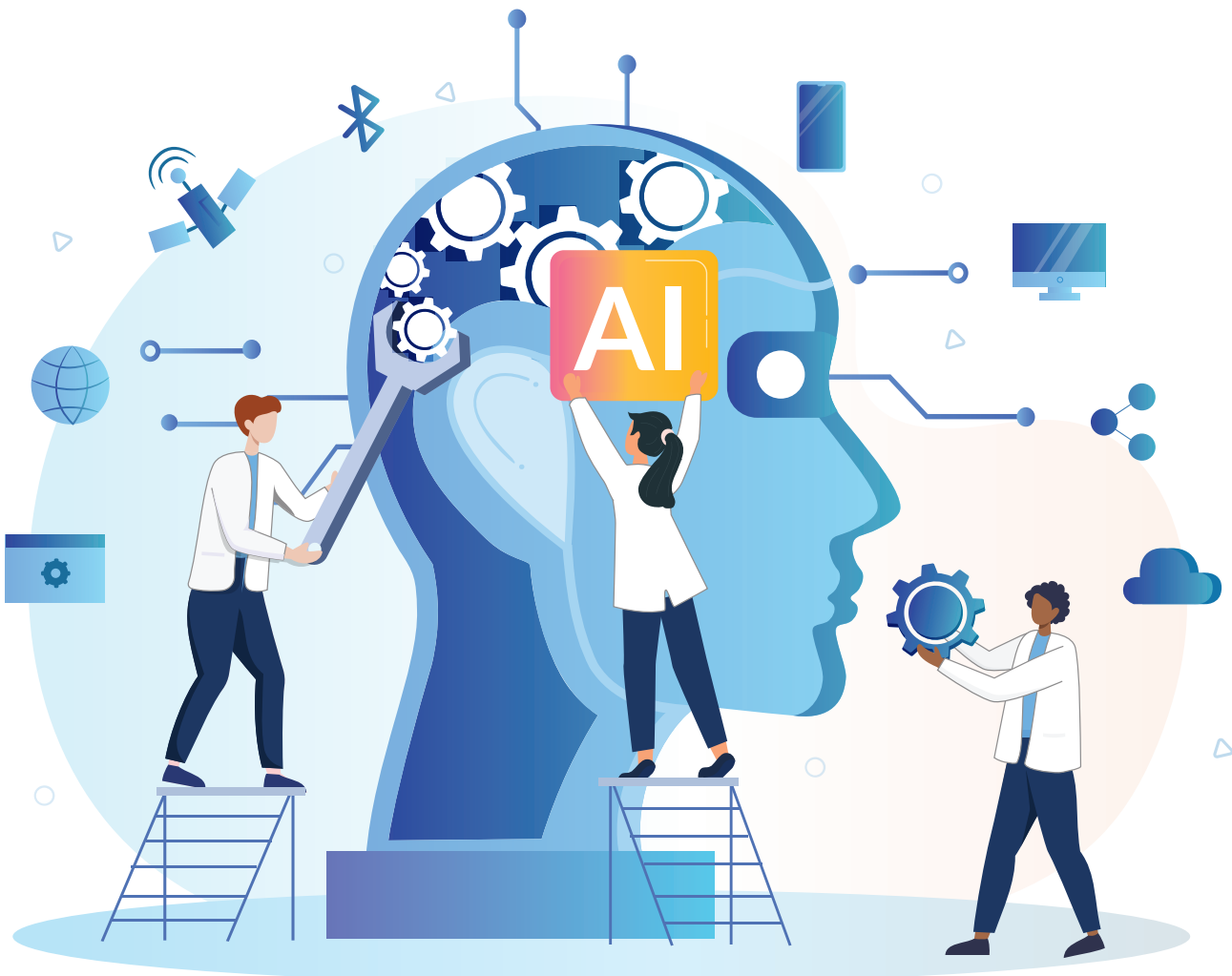


Neural Network Fundamentals



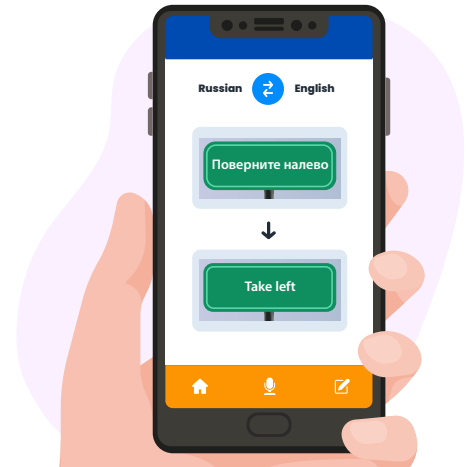
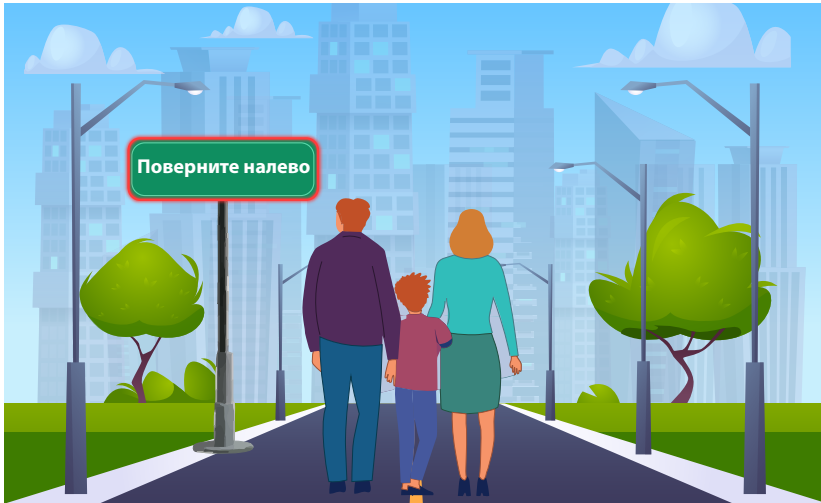
09

Artificial Intelligence (AI), powered by neural networks, is vital today as it drives innovations across industries, enabling smarter decisions, personalized experiences, and automation of complex tasks.



In today's rapidly evolving technological world, advancements are helping to solve various everyday problems. From navigating unfamiliar cities to translating languages in real-time, technology simplifies our lives.

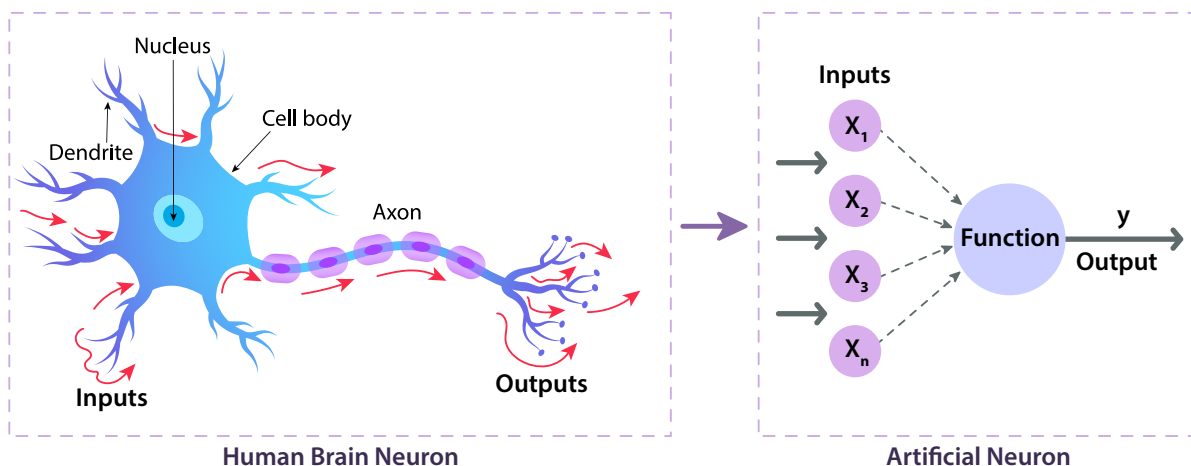
For instance, imagine you are travelling in Russia and come across a noticeboard or signpost written in Russian, a language you do not understand. With Google's real-time translation feature, you can simply point your phone at the sign, and the text will instantly appear in English, making it easy to understand.



This real-time translation is one of many applications powered by neural networks. Neural networks form the backbone of numerous AI technologies, enabling machines to learn, make decisions, and perform tasks once thought impossible for computers.

Understanding Neural Networks

Neural networks are a class of Artificial Intelligence models designed to replicate the complex workings of the human brain. These models enable computers to learn from data and make pattern-based decisions. A neural network can be visualised as a highly complex web of interconnected units, called **nodes** or **neurons**, that collaborate to process and interpret information efficiently.

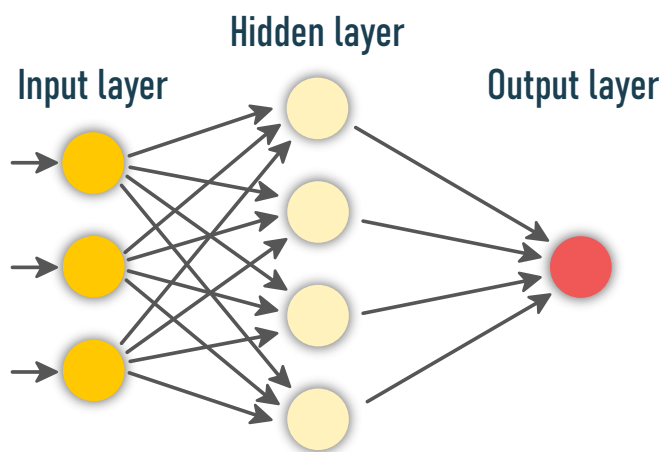


Neural networks consist of three primary components:

Input Layer: This layer serves as the entry point for data into the network, handling inputs such as images, text, or numerical data.

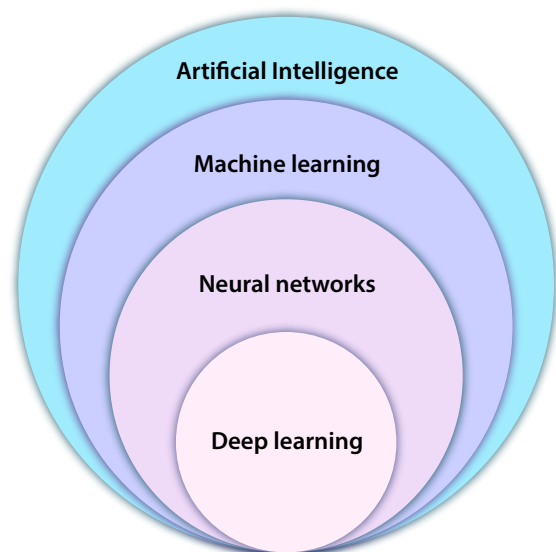
Hidden Layers: These intermediate layers are responsible for the core computations. They process and transform the input data through complex mathematical operations, identifying patterns, and extracting relevant features.

Output Layer: The final layer generates the output, producing a prediction or decision based on the processed information from the hidden layers.



Each node in the network processes the data and makes a decision based on it. As the network processes more data, it updates its parameters to enhance accuracy. This process, termed **training**, enables the network to recognise patterns and improve prediction precision over time.

Neural networks are a core component of Artificial Intelligence (AI). Within AI, they are employed in Machine Learning (ML) to make predictions based on data. **Deep learning**, a subset of ML, leverages neural networks with multiple layers to address more complex problems.



Artificial Neural Networks

Artificial Neural Networks, also referred to as **ANNs**, is a general term for all types of neural networks. These systems are inspired by the human brain, processing data through layers of interconnected nodes or neurons. ANNs encompass various types of networks, each designed for specific tasks.

Following are some types of Artificial Neural Networks.

1. Feedforward Neural Networks (FNN)
2. Convolutional Neural Networks (CNN)
3. Recurrent Neural Networks (RNN)
4. Generative Adversarial Networks (GANs)

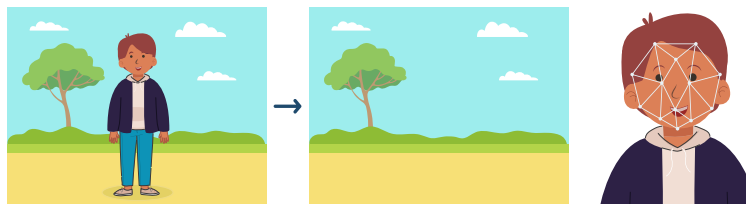
The hidden layers in various types of Artificial Neural Networks (ANNs) are structured differently according to the specific tasks they are designed to address. Each network type arranges its hidden layers to optimise performance for its particular application, such as image recognition, sequential data analysis, or data generation.

Applications of Artificial Neural Networks

Now that we have understood Artificial Neural Networks, let us also understand some of their primary applications.

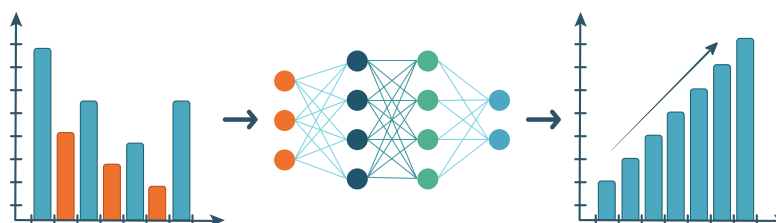
1. Face Recognition

Modern smartphone cameras utilise neural networks for tasks like age estimation based on facial features. Initially, the neural network distinguishes the face from its surrounding background. It then processes specific facial characteristics such as lines and textures to determine the individual's age.



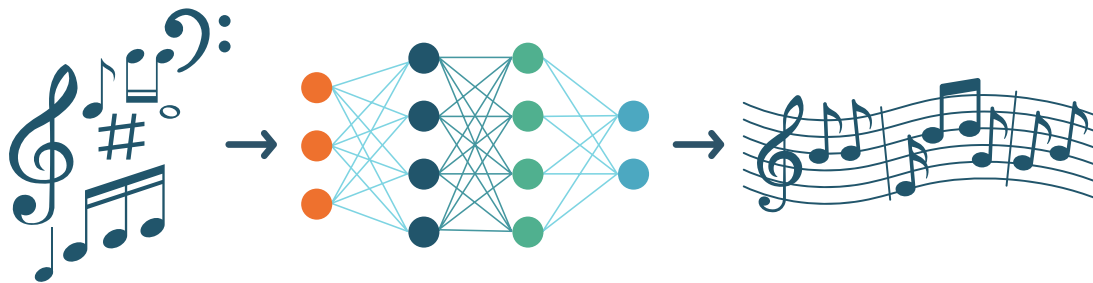
2. Prediction

Artificial Neural Networks (ANNs) forecast future events based on historical data. For instance, they can predict stock prices, weather patterns, or the probability of a customer making a purchase. By identifying and learning from past trends, neural networks generate accurate predictions about future outcomes.



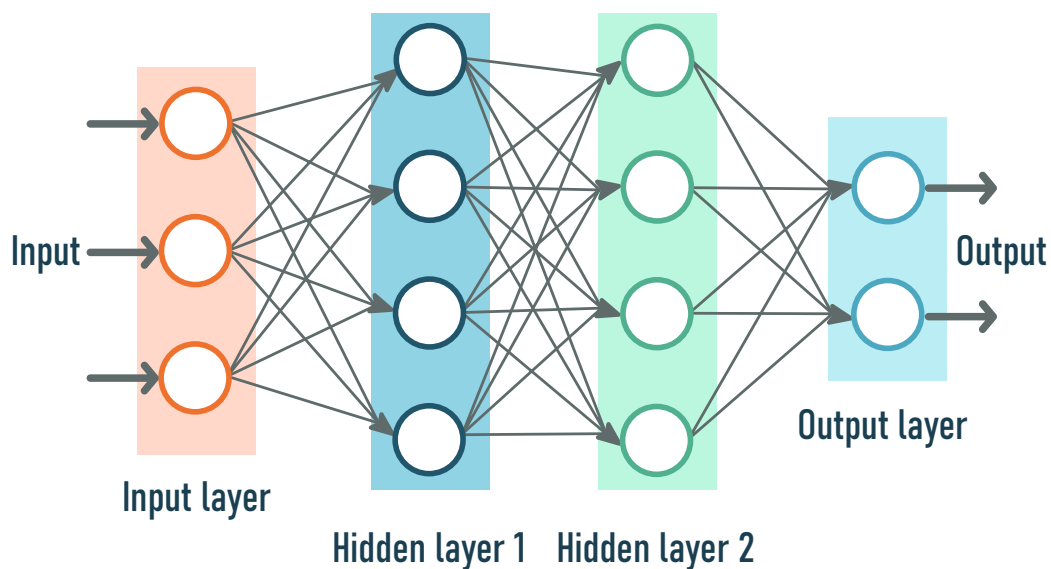
3. Music Composition

Artificial Neural Networks (ANNs) can generate music by learning from existing compositions. They analyse patterns in melody, rhythm, and harmony, enabling them to create new pieces that emulate different musical styles or genres.



To gain a practical understanding of neural networks, we will construct a basic Feedforward Neural Network (FNN) model. Before we begin, let us delve deeper into the workings and concepts of FNN.

Feedforward Neural Network (FNN): This is a type of artificial neural network in which data flows in a single direction, from the input layer to the output layer, passing through one or more hidden layers.



In Feedforward Neural Networks (FNNs), the input and output layers typically each consist of one layer. In contrast, the hidden layers can consist of one or more layers, depending on the complexity of the data and the problem being solved. Multiple hidden layers allow the network to capture more detailed patterns and relationships.

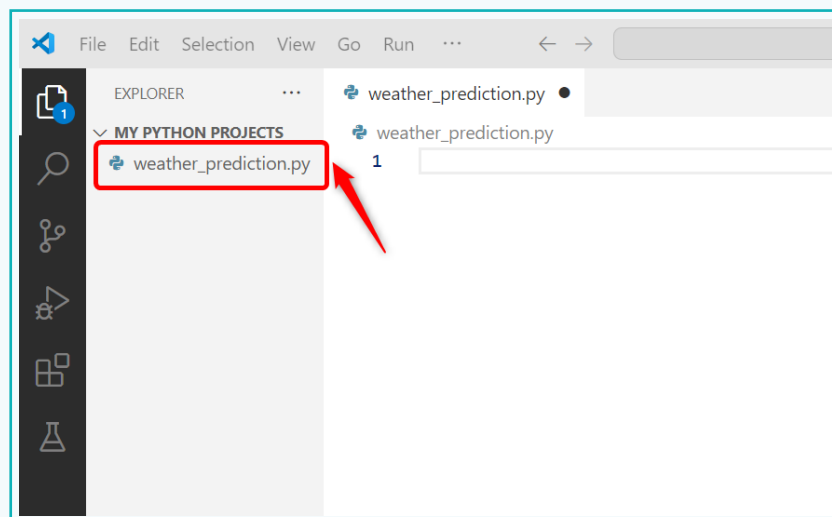
Activity

Let us create a simple Feedforward Neural Network model to predict rain based on basic weather data.

Part 1 - Initial Setup

1 Create a new folder in the desired location

- Open the VS Code editor, click on the 'File' menu, and select 'Open Folder'. Navigate to the newly created folder and click on 'Select Folder'.
- Once the folder appears in the VS Code editor's sidebar, create a new file within it and name it "weather_prediction.py".



Part 2 - Create a CSV File to Store the Data

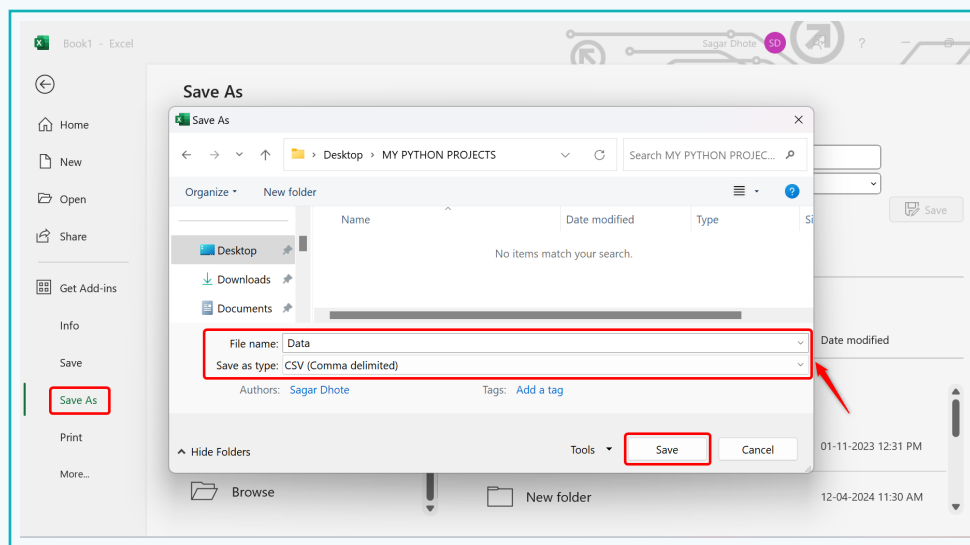
CSV File: A Comma Separated Values (CSV) file is a plain text file that contains a list of data separated by commas. CSV files are simple and easy to use, allowing data to be imported and exported between different programs.

- 1 Open Microsoft Excel and manually enter the data as shown.

	A	B	C
1	Temperature	Humidity	RainTomorrow
2	22	80	1
3	25	70	0
4	21	85	1
5	24	65	0
6	20	90	1
7			

Activity

- Click on 'File' → 'Save As' and choose 'CSV' (comma delimited) from the 'Save as type' dropdown menu.
- Name the file "Data.csv" and save it in the same folder where 'weather_prediction.py' is stored.



The 'Data.csv' file will now appear in the VS Code project folder.

Part 3 - Prepare Data for the Model

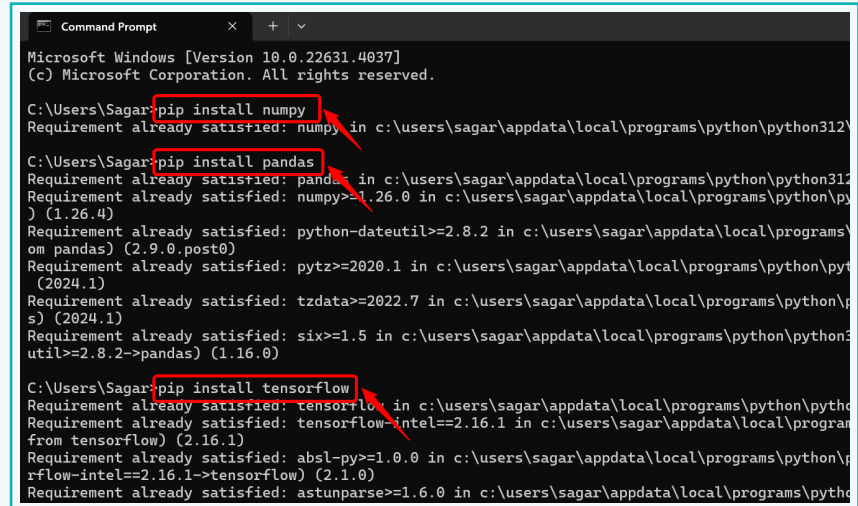
To build a neural network model, specific libraries are required for data handling. For this, we will use Pandas, Numpy, and TensorFlow.

- Pandas:** It is used for handling and analysing data in Python, particularly for loading, cleaning, and organising the data.
- Numpy:** It is a fundamental library used for numerical computations in Python. It is used for creating and manipulating arrays, performing mathematical operations, and handling large datasets efficiently.
- TensorFlow:** It is a powerful library used for building and training neural networks, including the Feedforward Neural Network (FNN) model.

Activity

- 1 Open the Command prompt and pass the following commands to install the required modules.

- `pip install pandas`
- `pip install tensorflow`
- `pip install numpy`



```
Microsoft Windows [Version 10.0.22631.4037]
(c) Microsoft Corporation. All rights reserved.

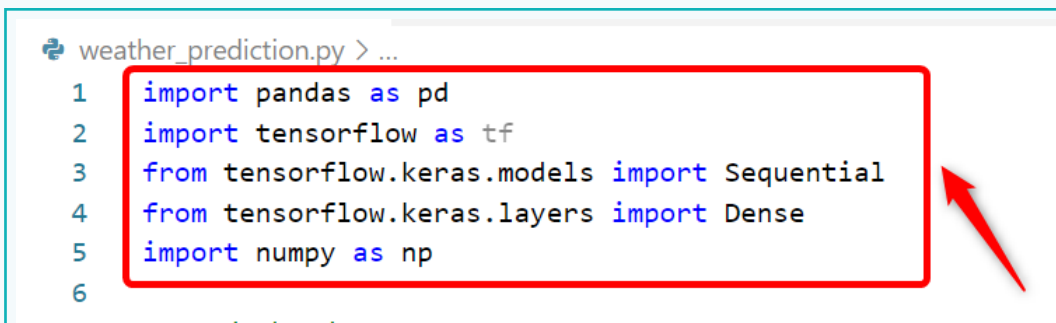
C:\Users\Sagar>pip install numpy
Requirement already satisfied: numpy in c:\users\sagar\appdata\local\programs\python\python312\
...
C:\Users\Sagar>pip install pandas
Requirement already satisfied: pandas in c:\users\sagar\appdata\local\programs\python\python312\
Requirement already satisfied: numpy>=1.26.0 in c:\users\sagar\appdata\local\programs\python\py
...
C:\Users\Sagar>pip install tensorflow
Requirement already satisfied: tensorflow in c:\users\sagar\appdata\local\programs\python\pytho
Requirement already satisfied: tensorflow-intel==2.16.1 in c:\users\sagar\appdata\local\progra
...
Requirement already satisfied: astunparse>=1.6.0 in c:\users\sagar\appdata\local\programs\pytho
```

Note:

If the modules are already installed on your system, skip the above step.

- 2 After installing the modules, import the required libraries in the 'weather_prediction.py' file.

- Import *pandas* with the alias *pd*.
- Import *TensorFlow* as *tf*.
- Import *Sequential* from *tensorflow.keras.models* for creating a neural network model.
- Import *Dense* from *tensorflow.keras.layers* for adding fully connected layers to the model.
- Import *numpy* with the alias *np*.



```
weather_prediction.py > ...
1 import pandas as pd
2 import tensorflow as tf
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import Dense
5 import numpy as np
6
```

Activity

- 3 Now, load the data from the CSV file and save it in the Pandas DataFrame.
- Use the `read_csv` function from the `pandas` module to load the CSV file data. Save the data in a `pandas DataFrame` and print it to the terminal using the `print()` function.

```
5
6 # Load the dataset
7 data = pd.read_csv('Data.csv')
8 print(data)
9
```

Output -

```
>>> data = pd.read_csv('Data.csv')
>>> print(data)
```

	Temperature	Humidity	RainTomorrow
0	22	80	1
1	25	70	0
2	21	85	1
3	24	65	0
4	20	90	1

```
>>>
```

As we can observe, in the data, there are two features— 'temperature' and 'humidity' as input parameters and a 'target' variable 'RainTomorrow' as an output parameter.

- 4 Now, extract the input features and the target variable from the DataFrame to two separate variables X and y.
- Extract the columns 'Temperature' and 'Humidity' from the DataFrame and convert them into a NumPy array using the `values()` function. Thus, array x contains the input features for the model.
 - Extract the column `RainTomorrow` from the DataFrame and convert it into a NumPy array using the `values()` function. Thus, array y contains the target variable we want to predict.

```
9
10 # Prepare features and target
11 X = data[['Temperature', 'Humidity']].values
12 y = data['RainTomorrow'].values
13
```

Activity

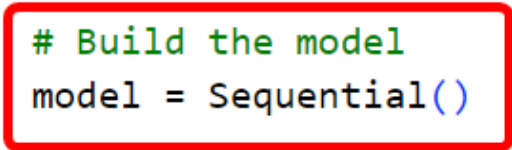
Note:

Converting DataFrame columns into NumPy arrays is necessary for model training and evaluation in TensorFlow. This conversion ensures that the data is in a format compatible with the neural network.

Part 4 - Create the Model

- 1 Create a *'model'* variable and pass *Sequential()* to it. This sets up the basic structure of our neural network, where we will add layers one by one.

```
13
14 # Build the model
15 model = Sequential()
16
```



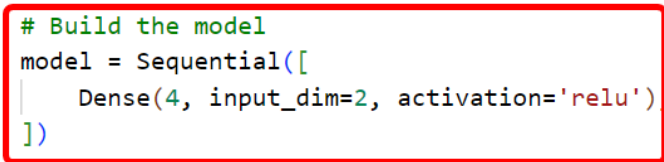
Here,

Sequential(): It initialises a model where we can stack layers on top of each other in a sequence.

2 Add the First Layer

- Next, add a *Dense* (fully connected) layer to the model. This layer will have 4 neurons and will take input with 2 features.
- The number of neurons is based on the data's complexity and the model's performance.
- The *ReLU* activation function is used to help the model learn complex patterns.

```
--
14 # Build the model
15 model = Sequential([
16     Dense(4, input_dim=2, activation='relu') # Input + hidden layer
17 ])
18
```



Activity

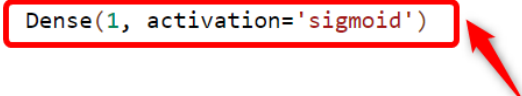
Here,

- **Dense(4, ...):** This means the layer has four neurons. Each neuron in this layer is connected to every input feature.
- **input_dim=2:** This tells the model that the input data has two features (e.g., temperature and humidity).
- **activation='relu':** The *ReLU* (Rectified Linear Unit) activation function helps the model learn by allowing only positive values to pass through while converting negative values to zero.

3 Add the Output Layer

- Add a dense layer with one neuron. This layer determines whether it will rain tomorrow or not. We use the *Sigmoid* activation function to get a probability between 0 and 1.

```
14 # Build the model
15 model = Sequential([
16     Dense(4, input_dim=2, activation='relu'), # Input + hidden layer
17     Dense(1, activation='sigmoid')           # Output layer
18 ])
19
```



Here,

- **Dense(1, ...):** This is the output layer with one neuron. It produces a single value as an output.
- **activation='sigmoid':** The Sigmoid function maps the output between 0 and 1, which is useful for making a yes/no (binary) prediction.

Part 5 - Compile and Train the Model

The next step is to train the model. However, we need to compile it first. Compiling prepares the model for training by specifying how it will learn from the data.

- 1 Set up the model for training by specifying how it should learn. Choose an optimizer, loss function, and performance metrics.

```
19
20 # Compile the model
21 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
22
```



Activity

Here,

- **optimizer='adam'**: Adjusts the model's weight during training to minimise errors and improve performance.
- **loss= 'binary_crossentropy'**: Measures how well the model's predictions match the actual outcomes. For binary classification (yes/no), this function helps evaluate errors.
- **metrics= ['accuracy']**: Tracks how often the model's predictions are correct, providing a measure of overall performance.

- 2 Train the model to make predictions by adjusting its weights based on the data. This is done through multiple iterations where the model learns and improves.

```
22  
23 # Train the model  
24 model.fit(X, y, epochs=10, batch_size=1, verbose=1)  
25
```

Here,

- **X**: The input features (e.g., temperature and humidity)
- **y**: The target variable (e.g., whether it will rain tomorrow)
- **epochs=10**: The number of times the model will go through the entire dataset during training
- **batch_size=1**: How many samples the model looks at before updating its weights. A batch size of 1 means the model updates after each sample
- **verbose=1**: Shows progress during training so you can see how the learning is going

Part 6 - Test the Model

Now, test the trained model to predict weather conditions.

- 1 Create a new variable named *input_features*. Pass two values for temperature and humidity. These are the new inputs for which we want to predict the outcome.

```
26  
27 #Prediction  
28 input_features = [25, 60]  
29
```

Activity

- 2 Convert the list to a NumPy array. NumPy arrays are used because they are compatible with the model's prediction function.

```
26
27 #Prediction
28 input_features = np.array([[25, 60]])
29
```



- 3 Now, create a variable *predictions*. Use the model's predict method to get the predictions. Pass the *input_features* to the model to get its predictions based on the new data. Print the 'predictions' using the *print()* function.

```
29
30 #Feed the input features into the model
31 predictions = model.predict(input_features)
32 print(predictions)
33
```



Output -

```
>>> input_features = np.array([[25, 60]])
>>> predictions = model.predict(input_features)
1/1 ————— 0s 43ms/step
>>> print(predictions)
[[0.99999994]]
>>>
```



The model's output is a probability between 0 and 1, which represents the probability of an event (here, rain) occurring. To convert this into a clear **yes** or **no** prediction (0 or 1), we check if the probability is greater than 0.5. If it's greater than 0.5, we predict 1 (indicating rain); otherwise, we predict 0 (no rain).

Activity

- 4 Create a new variable *predicted_class*. Store the binary output (0 or 1) based on the threshold.
- Here, **(predictions > 0.5)** converts probabilities into a Boolean array, where each value is True if it is greater than 0.5, otherwise False.
 - .astype(int)** converts the Boolean array into integers, where True becomes 1 and False becomes 0.

```
30 #Feed the input features into the model
31 predictions = model.predict(input_features)
32 predicted_class = (predictions > 0.5).astype(int)
33
```

Here, '1' represents the possibility that it will rain, while '0' represents the possibility that it will not rain.

- 5 Finally, print the *predicted_class* using the *print()* function. '*predicted_class[0][0]*' accesses the predicted value from the *predicted_class* array.

```
33
34 print('Prediction:', predicted_class[0][0])
35
36
```

Output -

```
Epoch 10/10
5/5 _____ 0s 4ms/step - accuracy: 0.1639 - loss: 2.8937
1/1 _____ 0s 78ms/step
Prediction: 1
PS D:\MY PYTHON PROJECTS> 
```

The output is '1', which indicates that it will rain.

You can go back to *input_features* and experiment with different humidity and temperature variables to observe how the output changes.

Note:

The output of this model may not be highly accurate because the data used to train the model was very limited. In general, the larger the dataset used for training, the more accurate the model's predictions will be.

In this lesson, we explored Neural Networks, including their structure, functionality, and real-world applications. We built a weather prediction model using a Feedforward Neural Network and employed core libraries such as Pandas, TensorFlow, Keras, and NumPy for data handling, model building, and numerical computations, ensuring efficient neural network implementation.

Exercise

Fill in the Blanks.

- 1 Neural networks mimic the way our brain works and consist of interconnected _____ or _____ that work together to process information.
- 2 Neural networks have three main parts: the _____ layer, hidden layers, and the _____ layer.
- 3 The _____ is a powerful library used for building and training neural networks.

State if the statements are True or False.

- 4 Neural networks use real-time translation features to help us understand text in different languages.
▲ _____
- 5 The Feedforward Neural Network (FNN) can move data in multiple directions, both forward and backward.
▲ _____
- 6 Artificial Neural Networks (ANNs) forecast future events based on historical data.
▲ _____

Tech Flash

- | | | |
|---|---|--|
| Neural Network | : | A type of artificial intelligence that mimics the human brain's structure. It helps computers to learn and make decisions through interconnected nodes or neurons. |
| Feedforward Neural Network (FNN) | : | A type of neural network where data flows in a single direction from input to output through one or more hidden layers. |
| Pandas | : | A Python library used for efficient data manipulation, including loading, cleaning, and organising. |
| NumPy | : | A fundamental Python library for numerical operations, enabling the creation and manipulation of arrays and performing mathematical calculations. |